

React & TypeScript

Un cours condensé pour apprendre à construire des interfaces web modernes avec React, typées et robustes grâce à TypeScript.

 **React**
18

 **TypeScript**
5

1 Qu'est-ce que React & TypeScript ?

Comprendre l'écosystème avant d'écrire son premier composant.

React, la bibliothèque UI

React est une bibliothèque JavaScript créée par Meta pour construire des interfaces déclaratives, basées sur des **composants** réutilisables. L'interface se met à jour automatiquement quand les données (le *state*) changent.

TypeScript, le sur-ensemble typé

TypeScript ajoute un système de types statique à JavaScript. Les erreurs de type sont détectées à la compilation plutôt qu'à l'exécution, ce qui améliore la fiabilité et l'auto-complétion dans l'éditeur.

Composants réutilisables

JSX

Typage statique

Virtual DOM

Créer un projet

- 1 Installer **Node.js** (LTS) depuis nodejs.org.
- 2 Créer un projet avec Vite : `npm create vite@latest mon-app -- --template react-ts`
- 3 Installer les dépendances : `cd mon-app && npm install`
- 4 Lancer le serveur de développement : `npm run dev`

```
$ npm run dev VITE ready – http://localhost:5173
```

JSX en un coup d'œil

```
function App() {  
  return (  
    <div className="app">  
      <h1>Bonjour React !</h1>  
    </div>  
  );  
}
```

2 Types, interfaces & génériques

Le strict minimum de TypeScript pour être productif avec React.

Types de base

```
let age: number = 25;
let nom: string = "Alice";
let actif: boolean = true;
let tags: string[] = ["a", "b"];

// Union de types
let id: string | number;
```

Objets & interfaces

```
interface Utilisateur {
  id: number;
  nom: string;
  email?: string; // optionnel
}

const u: Utilisateur = {
  id: 1,
  nom: "Sam",
};
```

Fonctions typées

```
function addition(a: number, b: number): number {
  return a + b;
}

// Type alias pour une fonction
type Callback = (valeur: string) => void;
```

Génériques

```
function premier<T>(liste: T[]): T {
  return liste[0];
}

premier<number>([1, 2, 3]); // T = number
premier(["a", "b"]);       // T inféré = string
```

ASTUCE

Dans la grande majorité des cas, laissez TypeScript **inférer** le type automatiquement plutôt que de l'annoter partout — n'annotez que les paramètres de fonction et les cas ambigus.

3 Composants & props typées

Un composant React est une fonction qui retourne du JSX.

Composant fonctionnel avec props typées

```
interface CarteProps {
  titre: string;
  description: string;
  onValider?: () => void; // callback optionnel
}

function Carte({ titre, description, onValider }: CarteProps) {
  return (
    <div className="carte">
      <h2>{titre}</h2>
      <p>{description}</p>
      <button onClick={onValider}>Valider</button>
    </div>
  );
}

// Utilisation
<Carte titre="Offre" description="Détails de l'offre" />
```

Composition & children

```
interface PanneauProps {
  children: React.ReactNode;
}

function Panneau({ children }: PanneauProps) {
  return <div className="panneau">{children}</div>;
}
```

Rendu conditionnel & listes

```
{estConnecte && <p>Bienvenue !</p>}
{erreur ? (
  <p>{erreur}</p>
) : (
  <p>Tout va bien</p>
)}
```

```
{produits.map((p) => (
  <li key={p.id}>
    {p.nom}
  </li>
))}
```

PIÈGE FRÉQUENT

Oublier la prop key unique sur chaque élément d'une liste provoque des avertissements et des bugs de rendu lors des mises à jour.

4 useState & useEffect

Les deux hooks que vous utiliserez le plus souvent.

useState : état local

```
import { useState } from "react";

function Compteur() {
  const [valeur, setValeur] = useState<number>(0);

  return (
    <button onClick={() => setValeur(valeur + 1)}>
      Valeur : {valeur}
    </button>
  );
}
```

RÈGLE D'OR

Ne jamais modifier le state directement (`valeur++`) : toujours passer par le setter (`setValeur(...)`) pour déclencher un nouveau rendu.

useEffect : effets de bord

```
import { useState, useEffect } from "react";

function Horloge() {
  const [heure, setHeure] = useState(new Date());

  useEffect(() => {
    const id = setInterval(() => setHeure(new Date()), 1000);
    return () => clearInterval(id); // nettoyage
  }, []); // tableau vide = exécuté une seule fois

  return <p>{heure.toLocaleTimeString()}</p>;
}
```

Tableau de dépendances	Comportement
Absent	Exécuté après chaque rendu
[]	Exécuté une seule fois (montage du composant)
[valeur]	Exécuté quand valeur change

5 Context, reducer & hooks personnalisés

Partager de l'état entre composants sans tout faire remonter manuellement.

useContext : partager un état global

```
interface ThemeContextType { sombre: boolean; bascule: () => void; }

const ThemeContext = createContext<ThemeContextType | null>(null);

function useTheme() {
  const ctx = useContext(ThemeContext);
  if (!ctx) throw new Error("useTheme hors provider");
  return ctx;
}
```

useReducer : logique d'état complexe

```
type Action = { type: "incrémenter" } | { type: "reset" };

function reducer(etat: number, action: Action): number {
  switch (action.type) {
    case "incrémenter": return etat + 1;
    case "reset": return 0;
  }
}

const [etat, dispatch] = useReducer(reducer, 0);
dispatch({ type: "incrémenter" });
```

Hook personnalisé

```
function useLocalStorage(cle: string, valeurDefault: string) {
  const [valeur, setValeur] = useState(
    () => localStorage.getItem(cle) ?? valeurDefault
  );
  useEffect(() => localStorage.setItem(cle, valeur), [cle, valeur]);
  return [valeur, setValeur] as const;
}
```

ASTUCE

Un hook personnalisé n'est qu'une fonction qui commence par use et appelle d'autres hooks — le moyen idéal de réutiliser de la logique d'état entre composants.

6 Gérer les entrées utilisateur

Typier correctement les événements DOM avec TypeScript.

Champ contrôlé

```
function Formulaire() {
  const [nom, setNom] = useState("");

  const gererChangement = (e: React.ChangeEvent<HTMLInputElement>) => {
    setNom(e.target.value);
  };

  const gererEnvoi = (e: React.FormEvent) => {
    e.preventDefault();
    console.log("Envoyé :", nom);
  };

  return (
    <form onSubmit={gererEnvoi}>
      <input value={nom} onChange={gererChangement} />
      <button type="submit">Envoyer</button>
    </form>
  );
}
```

Types d'événements courants

Événement	Type TypeScript
Saisie dans un champ	<code>React.ChangeEvent<HTMLInputElement></code>
Clic sur un bouton	<code>React.MouseEvent<HTMLButtonElement></code>
Soumission de formulaire	<code>React.FormEvent<HTMLFormElement></code>
Touche clavier	<code>React.KeyboardEvent<HTMLInputElement></code>

ASTUCE

Laissez l'éditeur (VS Code) vous suggérer le bon type d'événement en tapant `onChange={ (e) => }` puis en survolant le paramètre `e`.

7 Navigation & appels API

Structurer une application à plusieurs pages et consommer une API typée.

React Router

```
import { BrowserRouter, Routes, Route, Link } from "react-router-dom";

function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Accueil</Link>
        <Link to="/profil">Profil</Link>
      </nav>
      <Routes>
        <Route path="/" element={ <Accueil /> } />
        <Route path="/profil" element={ <Profil /> } />
      </Routes>
    </BrowserRouter>
  );
}
```

Requête typée avec fetch

```
interface Utilisateur { id: number; nom: string; }

function ListeUtilisateurs() {
  const [utilisateurs, setUtilisateurs] = useState<Utilisateur[]>([]);

  useEffect(() => {
    fetch("/api/utilisateurs")
      .then((res) => res.json() as Promise<Utilisateur[]>)
      .then(setUtilisateurs);
  }, []);

  return <ul>{utilisateurs.map((u) => <li key={u.id}>{u.nom}</li>)}</ul>;
}
```

ALLER PLUS LOIN

Pour des applications plus complexes, **React Query** ou **SWR** gèrent le cache, le rechargement et les états de chargement/erreur automatiquement.

8 Bonnes pratiques & ressources

Les réflexes pour un code React/TypeScript propre et maintenable.

Organisation du projet

- Un composant par fichier, dossier `components/`
- Séparer logique métier (`hooks/`) et présentation
- Centraliser les types partagés dans `types.ts`

Tests

- **Vitest** ou **Jest** pour les tests unitaires
- **React Testing Library** pour tester le comportement, pas l'implémentation
- **Playwright** ou **Cypress** pour les tests end-to-end

Performance

- `React.memo` pour éviter des rendus inutiles
- `useMemo` / `useCallback` pour mémoriser calculs et fonctions
- Découper le code avec `React.lazy` et le *code splitting*

Qualité de code

- `eslint` + `typescript-eslint` pour le linting
- `prettier` pour un formatage cohérent
- Activer `strict: true` dans `tsconfig.json`

Prochaines étapes

Vous avez maintenant les bases pour construire une vraie application. Continuez avec :

- La documentation officielle : **react.dev** et **typescriptlang.org**
- Un framework full-stack comme **Next.js** pour le rendu serveur et le routing avancé
- Une librairie de composants : **shadcn/ui**, **MUI** ou **Chakra UI**
- Construire un petit projet complet : liste de tâches, tableau de bord, ou blog